

Architecture Report

Calculator Application

Himeur Idris – Diar Adem

2CS SIQ1 2025/2026

March 13, 2026

1. View 1 - Module View (Class Diagram)

The system is organized into two main packages: **calculator** and **interpreter**. The **Expression** interface is the core abstraction of the interpreter. **Number** and **Variable** represent terminal expressions, while **Plus**, **Minus**, and **Multiply** represent non-terminal expressions that combine other expressions. **Evaluator** parses a Reverse Polish Notation (RPN) string and builds an expression tree whose root is stored in **syntaxTree**.

In the calculator package, **MainGUI** is the presentation layer, **Calcul** is an adapter, **Trace** handles persistence, and **Operation** is the serializable data object stored in the trace file. **Calcul** is an adapter that converts calculator operations into RPN strings and delegates evaluation to **Evaluator**. **Trace** is intentionally invoked by the GUI rather than by the calculation layer to avoid coupling calculation logic with persistence concerns, preserving separation of responsibilities.

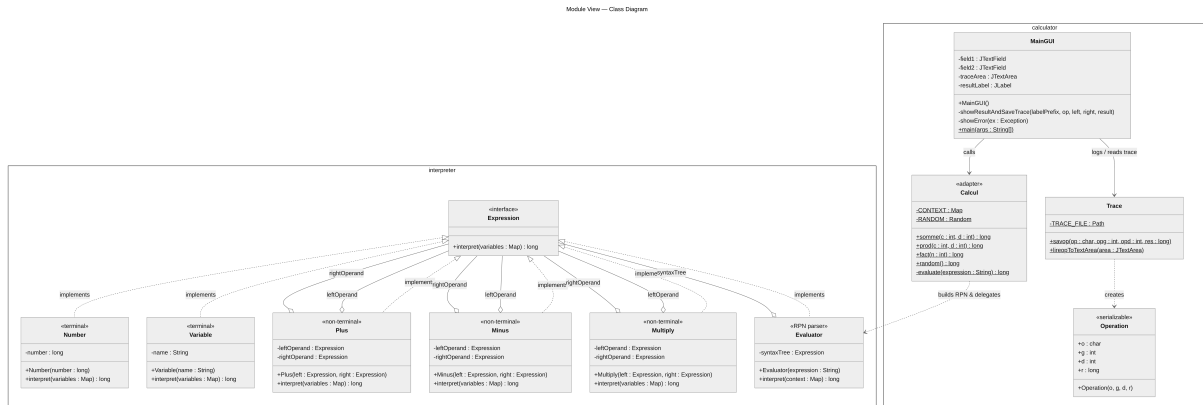


Figure 1: Module View – Class Diagram

2. View 2 - Component-and-Connector View

At runtime, the main interaction flow is **MainGUI** -> **Calcul** -> **interpreter** package. **MainGUI** collects user input, calls **Calcul**, receives the result, and displays it. **Calcul** builds an RPN string and delegates evaluation to the interpreter package. In this view, the interpreter is intentionally shown as a single component so the diagram emphasizes runtime connectors and package boundaries rather than internal implementation details.

A second connector exists between **MainGUI** and **Trace**. After an operation is completed, the

GUI records it directly through `Trace.savop(...)` and can later reload the history through `Trace.lireopToTextArea(...)`. This keeps persistence separate from computation and makes the layered architecture clearer: the calculator package uses the interpreter package, while the trace mechanism remains independent.

Component-and-Connector View — Development / Package Diagram

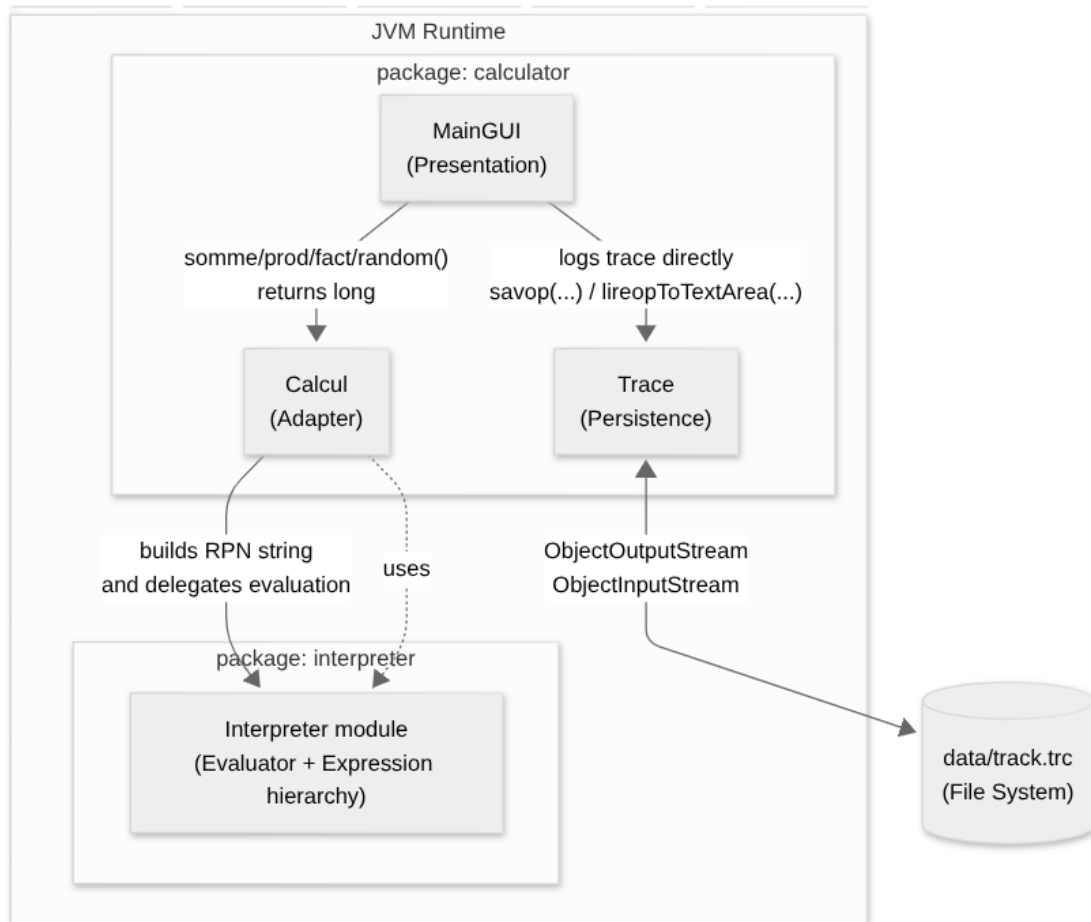


Figure 2: Component-and-Connector View

3. View 3 - Allocation View

The allocation view is intentionally minimal and focuses on runtime placement rather than internal implementation details. The main deployment elements are the user machine, the JVM execution environment, the calculator application artifact, the `interpreter.jar` artifact, and the trace file stored in the working directory.

Inside the JVM, the calculator application uses the interpreter JAR through in-process calls. The trace component accesses `data/track.trc` through read/write calls. This view highlights where the major runtime artifacts are allocated and how they interact, without repeating the internal structure already described in the module and component views.

Allocation View — Deployment Diagram

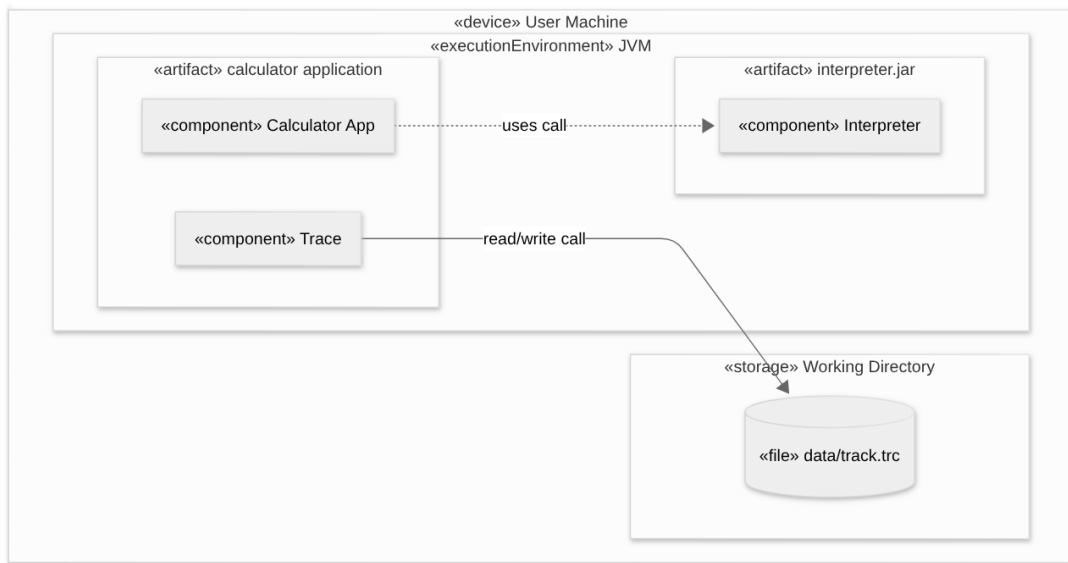


Figure 3: Allocation View – Deployment Diagram

4. GUI-Calculation Communication

The communication between the GUI and the calculation layer is synchronous and local to the same JVM. `MainGUI` calls methods such as `Calcul.somme(...)`, `Calcul.prod(...)`, `Calcul.fact(...)`, and `Calcul.random()`, and waits for the result immediately. The result is returned as a primitive `long` value, so no shared mutable state is exchanged between components. The GUI runs on the Swing Event Dispatch Thread, which means the connector is simply a direct procedure call rather than a network connector, IPC channel, or asynchronous message queue.